Application specific – access policies using Zscaler Private Access (ZPA)

Application specific – access policies using Zscaler Private Access (ZPA).

List of Team members:

S. No.	Name	College Name	Contact
1	Vekarnika B	Sri Ramakrishna College of Arts and Science for Women	srcw23218b48@srcw.ac.in
2	Advaita S S	Sri Ramakrishna College of Arts and Science for Women	srcw23218b02@srcw.ac.in
3	Dharshini J	Sri Ramakrishna College of Arts and Science for Women	srcw23218b15@srcw.ac.in
4	Sneha R	Sri Ramakrishna College of Arts and Science for Women	srcw23218b39@srcw.ac.in

Project Details

Title:

Application specific – access policies using Zscaler Private Access (ZPA).

Introduction:

In an era where the digital landscape is constantly evolving, organizations face an everincreasing threat from cyberattacks. The need for swift and effective incident response has become paramount to safeguard sensitive data, protect digital assets, and maintain the trust of stakeholders. In response to this growing challenge, we present a visionary project: the development of an AI-driven incident response platform that empowers cybersecurity teams with the tools to automate and enhance incident triage and response tasks.

In response to the evolving threat landscape and the dissolution of the traditional network perimeter, modern enterprises are rapidly shifting away from legacy VPN-centric remote access models. These conventional approaches often grant excessive network-level access, creating a significant attack surface and increasing the risk of lateral threat movement. To address these challenges, this project centers on the implementation of a Zero Trust security architecture, fundamentally operating on the principle of "never trust, always verify." By moving beyond location-based access, we aim to establish a security posture where trust is no longer implicit, and access is determined by granular context rather than network connectivity.

The core of this initiative involves the deployment of Zscaler Private Access (ZPA), a clouddelivered Zero Trust Access solution designed to create identity-aware, application-specific connectivity. Unlike traditional VPNs that place users on the internal network, ZPA leverages a "service-initiated" model to establish direct, one-to-one connections between authorized users and specific applications, completely hiding the corporate network and IP address space from the internet. This ensures that users connect exclusively to the applications they are permitted to access, based on continuous identity verification and context, rendering the applications invisible to unauthorized users and preventing common attack vectors such as port scanning and distributed denial-of-service (DDoS) attacks.

Ultimately, this project will enforce a least-privilege access model, ensuring users and devices are granted the minimum required access necessary to perform their functions. By decoupling application access from network access, we will eliminate the risk of lateral movement across the network, as users have no visibility or connectivity to other hosts. This strategic move not only significantly enhances security by reducing the attack surface but also simplifies the user experience and reduces operational overhead associated with legacy remote access infrastructure, positioning the organization at the forefront of secure and efficient access management.

Abstract

The proliferation of hybrid work and cloud-based resources has rendered traditional perimeterbased security models, such as VPNs, obsolete and increasingly dangerous. These legacy solutions often provide overly broad network access, exposing organizations to significant risks including lateral threat movement and an expanded attack surface. This project addresses these vulnerabilities by implementing a Zero Trust Architecture (ZTA) utilizing Zscaler Private Access (ZPA). The primary objective is to transition from network-centric to identity-centric access, thereby enforcing the principle of least privilege.

The methodology involves deploying ZPA's cloud-based service to create application-specific, identity-aware micro-tunnels. Unlike conventional VPNs that place users on the internal network, this solution establishes direct, one-to-one connections between verified users and authorized applications. The network infrastructure and all non-authorized applications remain completely invisible to the end-user, effectively eliminating surface for reconnaissance and attack. Access policies are dynamically governed by user identity, device posture, and contextual attributes, ensuring that connectivity is strictly limited to what is explicitly required for specific job functions.

The implementation of ZPA successfully decouples application access from network access, resulting in a demonstrable reduction in the organization's attack surface and the complete prevention of lateral movement. This project delivers a robust, scalable, and user-friendly remote access framework that aligns with modern security best practices. By shifting to an identity-aware, least-privilege model, the organization significantly enhances its security posture while simultaneously reducing operational complexity and improving the end-user experience.

Use Case Scenarios:

Use Case Scenarios 1:Secure Remote Employee Internet Access

An employee working from home needs secure access to the internet for daily work activities. Instead of directly connecting to the internet, the employee's traffic is routed through Zscaler. The employee authenticates using corporate credentials through Azure AD, which verifies their identity and applies security policies before allowing internet access.

Output:

- User identity verified: Employee_Group via Azure AD
- Secure authentication: Single Sign-On (SSO) enabled
- Security policy applied: Web filtering and malware inspection enabled
- Access granted: Internet access based on employee role
- Additional protection: Suspicious or malicious websites automatically blocked

Use Case Scenario 2: Secure Access to Cloud Applications

A finance department employee needs to access a cloud-based accounting application. When the employee attempts to log in, authentication is handled through Azure AD. Once the identity is verified, Zscaler allows secure access to the cloud application while inspecting traffic for potential threats.

Output:

- User identity verified: Finance_Department_Group
- Authentication method: Azure AD Single Sign-On
- Access granted: Limited to authorized financial applications
- Security enforcement: Data loss prevention (DLP) policies applied
- Threat protection: Traffic inspected for malware or suspicious activity

Use Case Scenario 3: Third-Party Vendor Access to Internal Application

A third-party vendor needs temporary access to a specific internal web application used for system maintenance. Instead of providing full network access, Zscaler restricts the vendor to only the required application. Authentication is performed using Azure AD, ensuring the vendor identity is verified.

Output:

- User identity verified: Vendor_User via Azure AD
- Access restriction: Limited to Maintenance_Application_Server
- Security enforcement: Time-based access policy enabled
- Network protection: No access to other internal resources
- Monitoring: All user activity logged for security auditing

Use Case Scenario 4: Secure Student Access to Educational Resources

A student accessing online learning materials from home needs secure access to the institution's learning portal. The authentication request is redirected to Azure AD, where the student logs in using institutional credentials. Zscaler then allows access only to approved educational resources.

Output:

- User identity verified: Student_Group via Azure AD
- Access allowed: Learning Management System (LMS) portal
- Security enforcement: Web filtering policies applied
- Restricted access: Blocked from administrative or staff resources
- Monitoring: Access activity recorded for security and compliance

Use Case Scenario 5: Administrator Access to Sensitive Systems

A system administrator needs to access critical internal servers for maintenance. Since the request involves sensitive infrastructure, Azure AD requires Multi-Factor Authentication before granting access. Zscaler ensures that the administrator can only access specific administrative systems.

Output:

- User identity verified: System_Admin_Group
- Security enforcement: Multi-Factor Authentication required
- Access granted: Limited to Infrastructure_Management_Server
- Context-aware policy: Access allowed only from approved devices
- Security monitoring: Administrative activity logged and monit

Stage – 1

Title:

Application specific – access policies using Zscaler Private Access (ZPA).

Overview:

In response to the evolving threat landscape and the dissolution of the traditional network perimeter, modern enterprises are rapidly shifting away from legacy VPN-centric remote access models. These conventional approaches often grant excessive network-level access, creating a significant attack surface and increasing the risk of lateral threat movement. To address these challenges, this project centers on the implementation of a Zero Trust security architecture, fundamentally operating on the principle of "never trust, always verify." By moving beyond location-based access, we aim to establish a security posture where trust is no longer implicit, and access is determined by granular context rather than network connectivity.

The core of this initiative involves the deployment of Zscaler Private Access (ZPA), a clouddelivered Zero Trust Access solution designed to create identity-aware, application-specific connectivity. Unlike traditional VPNs that place users on the internal network, ZPA leverages a "service-initiated" model to establish direct, one-to-one connections between authorized users and specific applications, completely hiding the corporate network and IP address space from the internet. This ensures that users connect exclusively to the applications they are permitted to access, based on continuous identity verification and context, rendering the applications invisible to unauthorized users and preventing common attack vectors such as port scanning and distributed denial-of-service (DDoS) attacks.

Ultimately, this project will enforce a least-privilege access model, ensuring users and devices are granted the minimum required access necessary to perform their functions. By decoupling application access from network access, we will eliminate the risk of lateral movement across the network, as users have no visibility or connectivity to other hosts. This strategic move not only significantly enhances security by reducing the attack surface but also simplifies the user experience and reduces operational overhead associated with legacy remote access infrastructure, positioning the organization at the forefront of secure and efficient access management.

Integrate Zscaler with Identity Providers (IdPs) such as Azure AD

List the team members:

S. No.	Name	College Name	Contact
1	Vekarnika B	Sri Ramakrishna College of Arts and Science for Women	srcw23218b48@srcw.ac.in
2	Advaita S S	Sri Ramakrishna College of Arts and Science for Women	srcw23218b02@srcw.ac.in
3	Dharshini J	Sri Ramakrishna College of Arts and Science for Women	srcw23218b15@srcw.ac.in
4	Sneha R	Sri Ramakrishna College of Arts and Science for Women	srcw23218b39@srcw.ac.in

List of Vulnerability table:

S.No.	Category Name	Vulnerability Name	CWE - No
1	Broken Access Control	Relative Path Traversal	CWE - 23
2	Security Misconfiguration	Active Debug Code	CWE – 489
3	Software Supply Chain Failures	Use of Unmaintained Third-Party Components	CWE – 1104
4	Cryptographic Failures	Weak Encoding for Password	CWE – 261
5	Injection	Improper Input Validation	CWE – 20
6	Insecure Design	Improper Privilege Management	CWE – 269
7	Authentication Failures	Use of Hard-coded Password	CWE – 259
8	Software or Data Integrity Failures	Improper Export of Android Application Components	CWE – 926
9	Security Logging and Alerting Failures	Information Loss or Omission	CWE – 221
10	Mishandling of Exceptional Conditions	Declaration of Catch for Generic Exception	CWE - 396

REPORT

1.

Vulnerability Name: Relative Path Traversal

CWE: CWE - 23

OWASP Category: A01:2025 Broken Access Control

Description: Relative Path Traversal (CWE-23) occurs when an application uses external input to construct file paths without properly validating or sanitizing the input. Attackers can insert special directory navigation sequences such as ../ to access files and directories outside the intended restricted location. Because operating systems automatically resolve these sequences during path processing, the application may unintentionally open, read, modify, or overwrite sensitive system files. This vulnerability typically appears in web applications that allow users to request files, upload files, or specify file paths dynamically.

For example, if a system expects a file like: /app/reports/report.pdf an attacker could manipulate the input to: ../../../../etc/passwd. The system resolves this path and accesses sensitive system files instead of the intended report directory.

Business impact: When this vulnerability exists in a production system, the consequences can reach far beyond a simple bug. Unauthorized data access may occur because attackers could read confidential files such as configuration files, database credentials, or user data. This can expose sensitive information and lead to further attacks. Data integrity risks arise if attackers overwrite or modify system files. They might replace configuration files, inject malicious scripts, or manipulate stored data used by the application.

System compromise becomes possible if attackers overwrite executable files, scripts, or libraries that the application later runs. That can lead to remote code execution, meaning the attacker effectively controls part of the system. Service disruption can also occur if important files are deleted or corrupted. This may crash the application or prevent it from starting, resulting in downtime and operational losses. From a business perspective, this can lead to data breaches, regulatory penalties, reputation damage, and service outages, especially in systems that handle customer information or financial data.

Example Language: **Perl**

```
my $dataPath = "/users/cwe/profiles";
my $username = param("user");
my $profilePath = $dataPath . "/" . $username;

open(my $fh, "<", $profilePath) || ExitError("profile read error: $profilePath");
print "<ul>\n";
while (<$fh>) {
  print "<li>$_</li>\n";
}
print "</ul>\n";
```

Recommendation:

1. Use Path Canonicalization: Normalize file paths using built-in functions such as realpath() in C/PHP, getCanonicalPath() in Java, or GetFullPath() in .NET before accessing the file system. This converts the path to its absolute form and removes traversal sequences.

2.Restrict Access to Specific Directories: Configure the application so that file operations are limited to a predefined directory (such as an upload or resource folder). The system should prevent access to files outside this directory.

3.Sanitize File Names: Remove or block directory separators (/ and \) and other unsafe characters from user-supplied filenames. Only allow safe characters and a limited number of dots in file names.

4.Allow Only Approved File Extensions: Use an allowlist of permitted file types (for example .txt, .pdf, .jpg) to prevent attackers from uploading or accessing dangerous files such as scripts or executables.

5.Avoid Direct Concatenation of User Input in Paths: Do not directly append user input to file paths when performing file operations. Instead, construct file paths using secure APIs and predefined base directories.

Vulnerability Name:

CWE: CWE - **OWASP**

Category:

2.

Active Debug Code

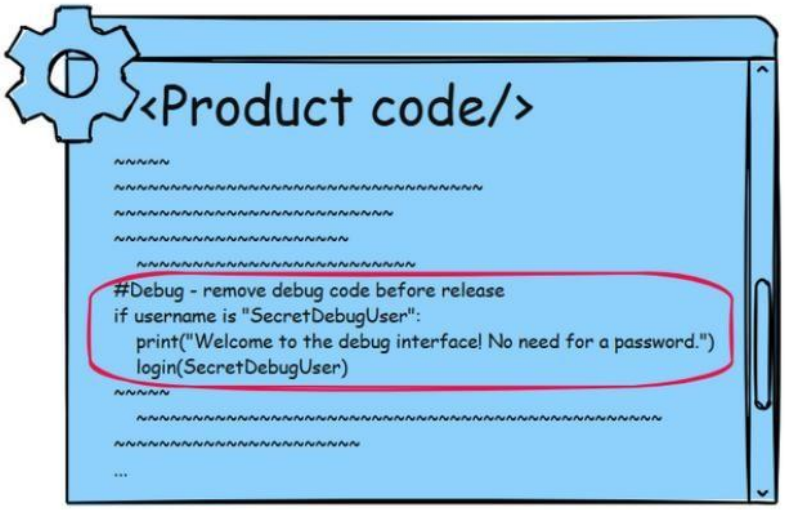
489

A02:2025 Security Misconfiguration

Description: Active Debug Code occurs when an application is released or deployed with debugging functionality still enabled. Debug code is typically added during development to test features, troubleshoot issues, or modify application behavior.

If this code is not removed before deployment, it may expose internal application details or create unintended entry points that attackers can exploit. In some cases, debug parameters or special conditions may bypass authentication checks, reveal configuration data, or allow administrative operations without proper authorization.

For example, a login system might contain a hidden parameter such as debug=1 that developers used during testing to skip authentication. If this parameter remains active in production, an attacker could manipulate the request to gain unauthorized access to the application**.**



Business Impact: Leaving debug code active in a production system can introduce serious security risks. Attackers may gain unauthorized access to restricted areas of the application if debug functions bypass authentication or security controls. This could allow them to impersonate users or administrators. Sensitive information such as configuration details, database queries, file paths, or system settings may be exposed through debugging messages. This information can help attackers understand the system architecture and plan further attacks.

Debug functionality may also provide unintended system control, enabling attackers to modify application behavior, execute administrative actions, or manipulate data stored within the system. In severe cases, exploitation of debug code may lead to complete compromise of the application or server, resulting in data breaches, service disruption, and reputational damage for the organization.

Example Language: **HTML**

```html
<FORM ACTION="/authenticate_login.cgi">
  <INPUT TYPE=TEXT name=username>
  <INPUT TYPE=PASSWORD name=password>
  <INPUT TYPE=SUBMIT>
</FORM>
```

Recommendation:

1. **Remove Debug Code Before Deployment:** Ensure all debugging statements, test interfaces, and temporary backdoor functions are removed before releasing the application to production.
2. **Disable Debug Mode in Production Environments:** Configure the application so that debug mode is disabled in production settings to prevent exposure of internal system information.
3. **Implement Secure Configuration Management:** Maintain separate configurations for development, testing, and production environments to ensure that debugging features are not accidentally enabled in live systems.
4. **Review Source Code for Debug Statements:** Conduct code reviews to identify and remove debugging functions, test parameters, and unnecessary logging that may expose sensitive information.
5. **Use Secure Logging Practices:** Replace debug outputs with controlled logging mechanisms that record necessary information without revealing sensitive system details.

Vulnerability Name:

CWE: CWE - **OWASP**

Category:

3.

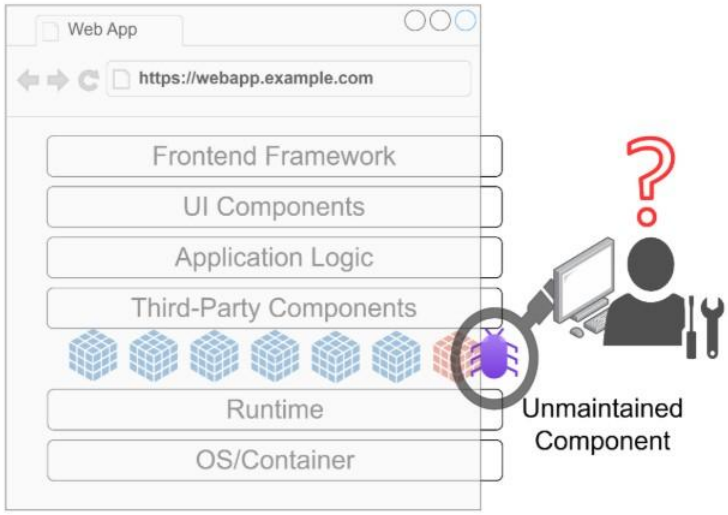
Use of Unmaintained Third-Party Components

1104

A03:2025 Software Supply Chain Failures

Description: This vulnerability occurs when an application uses third-party libraries, frameworks, or software components that are no longer maintained or supported by their developers. Because these components do not receive security updates or patches, any newly discovered vulnerabilities remain unfixed. Attackers can exploit these known weaknesses to compromise the application, gain unauthorized access, or execute malicious code. Using outdated or unmaintained components increases the risk of security breaches and can make the system vulnerable to multiple types of attacks. Proper dependency management and regular updates are necessary to ensure that all third-party components remain secure and supported.

Business Impact: Using unmaintained third-party components can expose the application to known security vulnerabilities that are no longer patched by developers. Attackers can exploit these weaknesses to gain unauthorized access, execute malicious code, or compromise sensitive data. This can lead to data breaches, service disruptions, and system compromise. For organizations, such incidents may result in financial losses, legal and regulatory penalties, damage to business reputation, and loss of customer trust, especially if critical systems or user information are affected.



Recommendation:

1. **Replace Unmaintained Components:** Identify third-party libraries or frameworks that are no longer supported and replace them with actively maintained and secure alternatives.
2. **Regularly Update Dependencies:** Keep all third-party components updated to their latest stable versions to ensure known vulnerabilities are patched.
3. **Use Dependency Monitoring Tools:** Implement tools that scan and monitor third-party libraries for known vulnerabilities and outdated versions.
4. **Maintain an Inventory of Components:** Keep a record of all third-party components used in the application to easily track updates and security advisories.
5. **Perform Regular Security Audits:** Conduct periodic security assessments and vulnerability scans to identify and address risks associated with external components.

Vulnerability Name:

CWE: CWE - **OWASP**

Category:

4.

Weak Encoding for Password

261

A04:2025 Cryptographic Failures

Description: Weak Encoding for Password occurs when an application stores or transmits passwords using weak encoding techniques rather than proper cryptographic protection. Developers sometimes attempt to “protect” passwords by encoding them using reversible methods such as Base64, hexadecimal encoding, or other trivial transformations.

Encoding is not encryption. It simply changes the representation of data and can easily be reversed by anyone who understands the encoding scheme. If an attacker gains access to configuration files, databases, or registry entries where encoded passwords are stored, they can decode the values and retrieve the original passwords. As a result, the application’s authentication mechanisms become vulnerable because the encoded password offers little to no real security protection.

Business Impact: Weakly encoded passwords can lead to serious security and operational risks for an organization. Attackers who obtain access to configuration files or system storage locations may easily decode stored passwords, allowing them to gain unauthorized access to databases, systems, or internal services. Once credentials are exposed, attackers may

impersonate legitimate users or administrators, enabling them to perform unauthorized actions such as modifying data, changing configurations, or accessing confidential information. Compromised credentials may also lead to privilege escalation, where attackers move from a normal user account to administrative control of the application or infrastructure. In severe cases, exposure of sensitive credentials can result in data breaches, financial losses, regulatory penalties, and reputational damage, particularly if customer or organizational data is affected.

Example Language: **Java**

```
...
Properties prop = new Properties();
prop.load(new FileInputStream("config.properties"));
String password = Base64.decode(prop.getProperty("password"));
DriverManager.getConnection(url, usr, password);
...
```

Recommendation:

1. Use Strong Cryptographic Hashing for Passwords: Store passwords using secure hashing algorithms such as bcrypt, PBKDF2, or Argon2 instead of reversible encoding methods.

2.Avoid Reversible Encoding for Credentials: Do not store passwords using simple encoding techniques such as Base64, hexadecimal encoding, or other easily reversible transformations.

3. Implement Salting Techniques: Add a unique random salt to each password before hashing to protect against rainbow table attacks and improve password security.

4.Protect Configuration Files and Credential Storage: Restrict access permissions to configuration files, registries, or databases where sensitive credentials are stored.

5.Use Secure Credential Management Systems: Store sensitive credentials in secure vaults or secret management systems rather than embedding them directly in application configuration files.

Vulnerability Name:

CWE: CWE - **OWASP**

Category:

Vulnerability Name:

CWE: CWE –

OWASP Category:

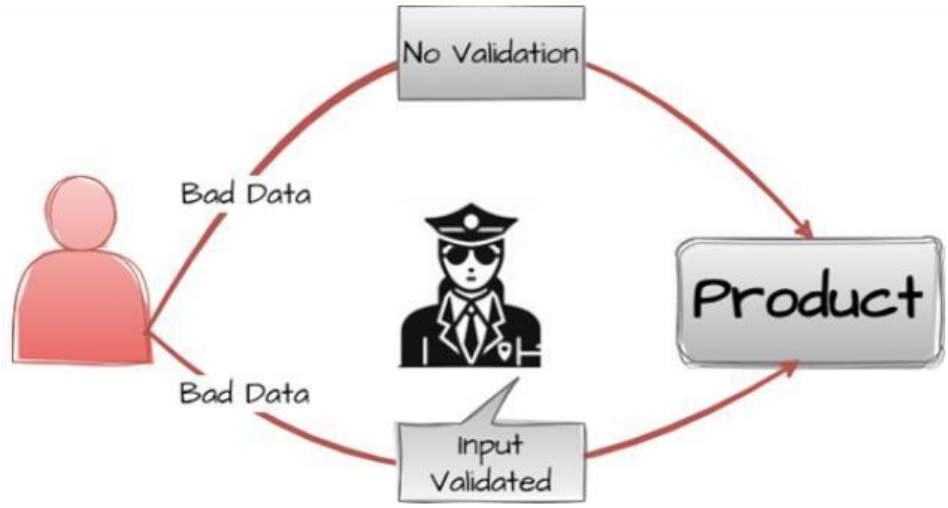
5.

Improper Input Validation

20

A05:2025 Injection

Description: Improper Input Validation occurs when a software application accepts input from users or external sources but fails to properly check whether the input meets the required format, type, length, or value range before processing it. When validation is missing or incorrect, attackers can provide malicious or unexpected data that the application processes without restriction. These inputs may come from forms, URL parameters, cookies, API requests, file uploads, or other external systems. Improper input validation can lead to serious security issues such as SQL Injection, Cross-Site Scripting (XSS), Command Injection, Buffer Overflow, and Path Traversal, allowing attackers to manipulate application behaviour, access sensitive data, or disrupt system operations.



Business Impact: Improper input validation can have significant impacts on an organization's security and operations. Attackers may exploit this vulnerability to manipulate application behaviour, access sensitive information, or execute unauthorized actions within the system. This can lead to data breaches, unauthorized modification of records, and disruption of services such as system crashes or denial-of-service attacks. In severe cases, attackers may gain control of application functions or execute malicious commands. These incidents can result in financial losses, regulatory or compliance issues, damage to the organization's reputation, and loss of customer trust.

Vulnerability Name:

CWE: CWE – **OWASP**

Category:

Example Language: **C** *(bad code)*

```
...
#define MAX_DIM 100
...
/* board dimensions */

int m,n, error;
board_square_t *board;
printf("Please specify the board height: \n");
error = scanf("%d", &m);
if ( EOF == error ){
  die("No integer passed: Die evil hacker!\n");
}
printf("Please specify the board width: \n");
error = scanf("%d", &n);
if ( EOF == error ){
  die("No integer passed: Die evil hacker!\n");
}
if ( m > MAX_DIM || n > MAX_DIM ) {
  die("Value too large: Die evil hacker!\n");
}
board = (board_square_t*) malloc( m * n * sizeof(board_square_t));
...
```

Recommendation:

1.Validate All Input Data: Ensure that all inputs from users or external sources are validated before processing. Check that the input matches the expected format, type, length, and range of values.

2.Implement Server-Side Validation: Always perform validation on the server side in addition to client-side validation, since client-side checks can be bypassed by attackers.

3.Use Input Whitelisting: Allow only predefined and acceptable input values instead of trying to block malicious inputs. This helps ensure that only valid data is processed by the application.

4.Limit Input Size and Range: Restrict the maximum length and acceptable range of input values to prevent resource exhaustion, buffer overflows, or unexpected system behaviour.

5.Conduct Regular Security Testing: Perform regular code reviews, vulnerability scans, and security testing to identify and fix input validation weaknesses during development.

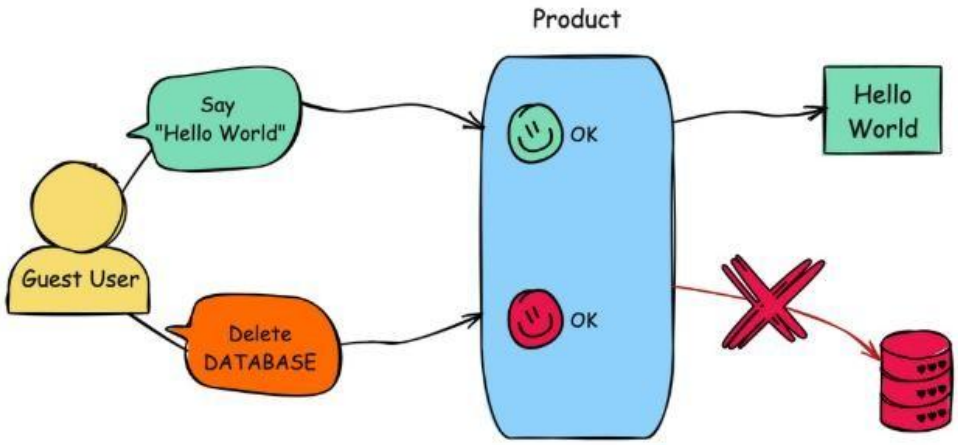
6

Improper Privilege Management

269

A06:2025 Insecure Design

Description: Improper Privilege Management occurs when a system does not properly assign, manage, modify, or verify user privileges within an application. Privileges define what actions a user or process is allowed to perform in a system. If these privileges are not correctly implemented or enforced, unauthorized users may gain access to restricted resources or perform actions beyond their intended permissions. This vulnerability can occur due to incorrect privilege assignment, failure to properly check user roles, or improper handling of elevated privileges during system operations. As a result, attackers may exploit the weakness to gain higher-level permissions, access sensitive information, or perform unauthorized administrative actions within the system.



Business Impact: Improper privilege management can lead to serious security and operational risks for organizations. Attackers or unauthorized users may gain elevated privileges and access sensitive system resources, confidential data, or administrative functions. This may result in unauthorized modification or deletion of critical data, disruption of system operations, and potential compromise of the entire application or infrastructure. In severe cases, attackers could take full control of system functions, leading to financial loss, regulatory noncompliance, damage to organizational reputation, and loss of customer trust.

Vulnerability Name:

CWE: CWE – **OWASP**

Category:

Example Language: **Java**

```
AccessController.doPrivileged(new PrivilegedAction() {
  public Object run() {
    // privileged code goes here, for example:
    System.loadLibrary("awt");
    return null;
    // nothing to return
  }
```

Recommendation:

1.Apply the Principle of Least Privilege: Assign users and system processes only the minimum level of access required to perform their tasks. Avoid granting unnecessary administrative privileges.

2.Implement Proper Access Control Checks: Ensure that the system verifies user roles and permissions before allowing access to sensitive operations or resources.

3.Use Role-Based Access Control (RBAC): Define clear roles such as administrator, operator, and user, and assign permissions based on these roles to ensure controlled access.

4.Monitor and Manage Privilege Changes: Track and log all privilege assignments, modifications, and usage to detect unauthorized privilege escalation attempts.

5.Perform Regular Security Reviews: Conduct periodic security audits, code reviews, and vulnerability assessments to identify and correct improper privilege management issues early.

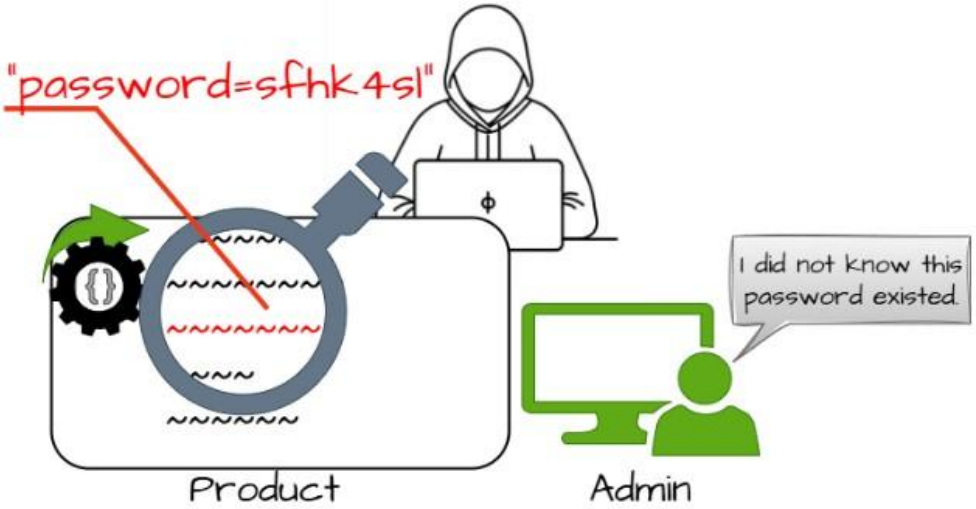
7.

Use of Hard-coded Password

259

A07:2025 Authentication Failures

Description: Use of Hard-coded Password occurs when a software application contains a password that is directly written into the source code or configuration files. These passwords are used for authentication either within the application itself or when connecting to external systems such as databases or services. Since the password is embedded in the code, anyone who gains access to the application code, binaries, or configuration files may be able to discover it. Hard-coded passwords are difficult to change after deployment and may remain the same across multiple installations of the software. This vulnerability can allow attackers to bypass authentication mechanisms, gain unauthorized access to systems, and perform actions with elevated privileges.



Business Impact: The use of hard-coded passwords can lead to serious security risks for organizations. If attackers discover the embedded password, they may gain unauthorized access to sensitive systems, databases, or administrative functions. This can result in data breaches, unauthorized modification or deletion of critical information, and compromise of system integrity. Because hard-coded passwords are difficult to change once the software is deployed, fixing the issue may require code updates or system reconfiguration. Such incidents can cause financial losses, regulatory compliance issues, operational disruptions, and damage to the organization's reputation and customer trust.

Vulnerability Name:

CWE: CWE – **OWASP**

Category:

Example Language: **C**

```c
int VerifyAdmin(char *password) {
  if (strcmp(password, "Mew!")) {
    printf("Incorrect Password!\n");
    return(0);
  }
  printf("Entering Diagnostic Mode...\n");
  return(1);
}
```

Example Language: **Java**

```java
int VerifyAdmin(String password) {
  if (!password.equals("Mew!")) {
    return(0);
  }
  //Diagnostic Mode
  return(1);
}
```

Recommendation:

1.Avoid Hard-coding Credentials: Never store passwords directly in source code, configuration files, or application binaries.

2.Use Secure Credential Storage: Store passwords in secure, encrypted configuration files, environment variables, or dedicated secret management systems.

3.Implement Strong Authentication Practices: Use hashed and salted passwords instead of storing them in plain text to protect sensitive authentication data.

4.Use Access Control and Role Management: Restrict access to systems and resources so that even if credentials are exposed, the impact is minimized.

5.Perform Regular Security Reviews: Conduct regular code reviews, vulnerability scans, and security audits to detect and remove hard-coded credentials from applications.

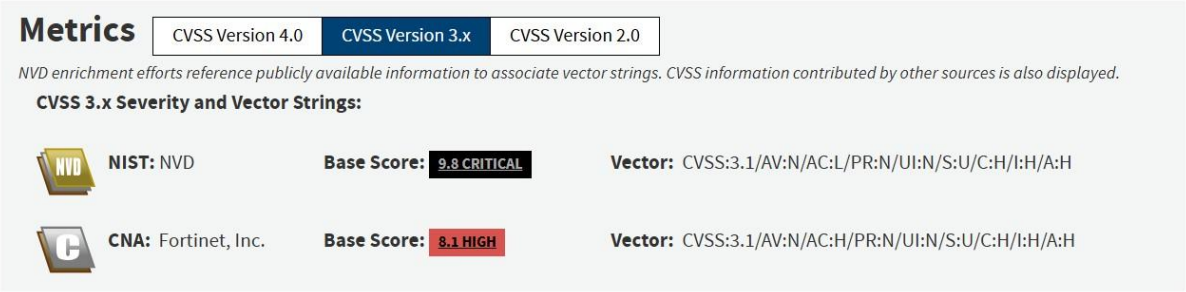
8.

Improper Export of Android Application Components

926

A08:2025 Software or Data Integrity Failures

Description: This vulnerability occurs when Android application components such as Activities, Services, Broadcast Receivers, or Content Providers are unintentionally exported and made accessible to other applications. When components are exported without proper access restrictions or permissions, malicious applications installed on the same device can interact with them. Attackers may exploit this misconfiguration to access sensitive data, trigger unintended actions, or manipulate application behavior. This usually happens due to incorrect settings in the AndroidManifest.xml file where the android: exported attribute is set improperly or not defined securely.



Business Impact: Improperly exported components can allow malicious applications to access sensitive application functionality or data without proper authorization. This may lead to exposure of user data, unauthorized actions within the app, or privilege escalation. Attackers could misuse the exported components to manipulate app operations or access confidential information. For organizations, this can result in data breaches, loss of user privacy, reputational damage, and loss of customer trust, especially if the application handles sensitive personal or financial information.

Vulnerability Name:

CWE: CWE – **OWASP**

Category:

Example Language: **XML** (bad code)

```
<activity android:name="com.example.vulnerableApp.mainScreen">

  ...
  <intent-filter>
    <action android:name="com.example.vulnerableApp.OPEN_UI" />
    <category android:name="android.intent.category.DEFAULT" />
  </intent-filter>
  ...
</activity>
<service android:name="com.example.vulnerableApp.backgroundService">

  ...
  <intent-filter>
    <action android:name="com.example.vulnerableApp.START_BACKGROUND" />
  </intent-filter>
  ...
</service>
```

Recommendation:

1.Restrict Exported Components: Explicitly define the android: exported attribute in the AndroidManifest.xml file and set it to "false" for components that should not be accessed by other applications. This prevents unauthorized apps from launching internal components.

2.Use Permission Protection: Apply specific permissions to exported components so that only trusted applications can access them. Developers can use permission attributes or signaturelevel permissions to restrict access to apps signed with the same certificate.

3.Limit Access to Content Providers: Configure proper read and write permissions for content providers and other shared components to ensure that only authorized applications can access sensitive data.

4.Reduce the Attack Surface: Export only the components that are absolutely necessary for external interaction. Any component not required for communication with other apps should remain internal to the application.

5.Perform Security Testing: Conduct regular mobile application security testing such as static code analysis, vulnerability scanning, and penetration testing to detect incorrectly exported components before deployment.

9.

Information Loss or Omission

221

A09:2025 Security Logging & Alerting Failures

Description: Information Loss or Omission occurs when a system fails to properly record or preserve security-relevant information during operations such as authentication, transactions, or system events. Logging and recording such information are essential for monitoring, auditing, and detecting suspicious activities. When security-related data is not recorded, is partially logged, or is improperly stored, important events may go unnoticed. This can happen when logs are truncated, important events are not captured, or warnings are not displayed to users. As a result, attackers may perform malicious actions without being detected, and administrators may not have enough information to investigate or respond to security incidents.

Business Impact: Information loss or omission can significantly affect an organization's ability to monitor and respond to security incidents. If critical events such as failed login attempts, suspicious activities, or system errors are not properly logged, attackers may operate undetected within the system. This lack of visibility can delay incident detection and investigation, increasing the risk of data breaches, unauthorized access, and system misuse. Additionally, incomplete or missing logs can make forensic analysis difficult and may lead to regulatory compliance issues, financial losses, operational disruptions, and damage to the organization's reputation.

Example Language: **PHP**

```php
function login($userName,$password){
  if(authenticate($userName,$password)){
    return True;
  }
  else{
    incrementLoginAttempts($userName);
    if(recentLoginAttempts($userName) > 5){
      writeLog("Failed login attempt by User: " . $userName . " at " + date('r') );
    }
  }
}
```

Vulnerability Name:

CWE: CWE – **OWASP**

Category:

Recommendation:

1.Implement Comprehensive Logging: Ensure that all security-relevant events such as authentication attempts, access to sensitive data, configuration changes, and system errors are properly logged.

2.Avoid Log Truncation or Omission: Configure logging systems to record complete information without truncating important details such as usernames, timestamps, IP addresses, or request parameters.

3.Protect and Maintain Log Integrity: Store logs securely and protect them from unauthorized modification or deletion to ensure reliable auditing and forensic analysis.

4.Implement Monitoring and Alerting: Use automated monitoring systems and alerts to detect suspicious activities or unusual patterns in system logs.

5.Conduct Regular Log Reviews: Periodically review logs and logging configurations to ensure that critical security events are being properly captured and analyzed.

10.

Declaration of Catch for Generic Exception

396

A10:2025 Mishandling of Exceptional Conditions

Description: Declaration of Catch for Generic Exception occurs when a program catches overly broad or generic exceptions such as Exception instead of handling specific exception types. While this approach may simplify code by reducing the number of catch blocks, it can hide important error conditions that require different handling. Catching generic exceptions may also unintentionally capture unexpected runtime errors that should be handled separately or allowed to propagate. As the application evolves and new exceptions are introduced, a generic catch block can prevent proper error handling and make it difficult to detect or troubleshoot security-related issues during execution.

Business Impact: Using generic exception handling can reduce the visibility of important system errors and security-related events. When specific exceptions are not properly handled, critical issues such as system failures, unauthorized activities, or data processing errors may be hidden within a generic error response. This can make troubleshooting, monitoring, and incident investigation more difficult. In some cases, attackers may exploit hidden errors to manipulate application behaviour or bypass security controls. These issues can lead to system instability, service disruptions, delayed incident detection, and potential financial or reputational damage to the organization.

Example Language: **Java**

```java
try {
  doExchange();
}
catch (IOException e) {
  logger.error("doExchange failed", e);
}
catch (InvocationTargetException e) {

  logger.error("doExchange failed", e);
}
catch (SQLException e) {

  logger.error("doExchange failed", e);
}
```

Vulnerability Name:

CWE: CWE – **OWASP**

Category:

Recommendation:

1.**Specific Exceptions:** Handle specific exception types rather than using generic exception classes such as Exception. This ensures that each error condition is handled appropriately.

2.Implement Proper Error Handling: Design clear and structured error-handling logic that responds correctly to different types of exceptions.

3.**Avoid Overly Broad Catch Blocks:** Do not use generic catch blocks unless absolutely necessary, and ensure that unexpected exceptions are properly logged and reviewed.

4.Log Detailed Error Information: Record detailed information about exceptions in system logs to support troubleshooting and security monitoring.

5.Perform Regular Code Reviews: Conduct code reviews and static analysis to identify and correct overly broad exception handling in the application code.

Stage – 2

Overview: -

An overview of a Nessus scan typically involves the following key steps and aspects:

1. Preparation: Define Objectives: Determine the scope and objectives of the scan. What systems or networks do you want to assess, and what specific vulnerabilities are you looking for?

   System Access: Ensure that you have the necessary permissions and access rights to scan the target systems or networks.

2. Configuration: Set Up Scan Policy: Create a scan policy that defines the parameters of the scan, such as the types of vulnerabilities to check for, scanning frequency, and specific configurations.

   Define Target Assets: Specify the IP addresses, hostnames, or network ranges of the systems you want to scan.

3. Scanning: Initiate Scan: Start the Nessus scan using the configured policy and target assets. Nessus will systematically scan the target systems for vulnerabilities based on its plugins and policies.

4. Vulnerability Identification: Nessus performs a comprehensive assessment, checking for vulnerabilities, misconfigurations, open ports, and other security issues. As the scan progresses, it categorizes vulnerabilities by severity, making it easier to prioritize remediation efforts.

5. Reporting: Nessus generates detailed reports that list the vulnerabilities discovered during the scan. Reports typically include information about the severity of each vulnerability, recommended actions for remediation, and supporting evidence. These reports can be customized and exported in various formats, such as PDF, HTML, or CSV.

6. Remediation: Review Findings: IT and security teams review the Nessus scan reports to prioritize and address the identified vulnerabilities.

   Remediation Steps: Take corrective actions to fix or mitigate the vulnerabilities. This may involve applying patches, changing configurations, or implementing security measures.

7. Follow-up: Regular Scanning: Conduct periodic Nessus scans to continuously monitor the security posture of the systems and networks. This helps identify new vulnerabilities and track progress in mitigating existing ones.

   Integration: Integrate Nessus with other security tools or workflows to automate the vulnerability management process and streamline the response to new findings.

8. Compliance Auditing (if applicable): If compliance checks are part of your scan policy, review compliance results and take actions to ensure systems meet the required standards and regulations.

9. Documentation: Maintain records of scan results, remediation efforts, and compliance checks for audit and reporting purposes.

10. User Training: Ensure that your IT and security teams are trained in using Nessus effectively and interpreting scan results.

Nessus scans are a critical component of proactive security, helping organizations identify and address vulnerabilities in their IT infrastructure to reduce the risk of security breaches and data compromises. The frequency and depth of scans can vary depending on an organization's security strategy and requirements.

Nessus is a popular vulnerability scanning tool used for identifying security weaknesses in computer systems and networks. It is widely utilized by security professionals and organizations to assess the security posture of their systems and to proactively identify potential vulnerabilities that could be exploited by attackers. Here's what I understand about Nessus scans:

1. Vulnerability Assessment: Nessus performs a vulnerability assessment by scanning target systems or networks to identify security issues, such as missing patches, misconfigurations, weak passwords, and other potential weaknesses that could be exploited by attackers.

2. Network Scanning: Nessus can scan a wide range of network devices, including servers, routers, switches, firewalls, and more. It examines network services, open ports, and software running on target systems.

3. Plugin Architecture: Nessus uses a plugin-based architecture, allowing it to check for a wide variety of vulnerabilities. These plugins are regularly updated to include the latest security checks and support for new vulnerabilities.

4. Compliance Auditing: It can also be used to assess systems against various compliance standards and regulatory requirements, helping organizations ensure that their systems meet specific security standards, like PCI DSS or CIS benchmarks.

5. Customization: Nessus scans can be customized to suit the specific needs of an organization. Users can define scan policies, select the target systems, and specify the types of vulnerabilities to scan for.

6. Reporting: Nessus provides detailed reports that highlight vulnerabilities, their severity, and remediation recommendations. These reports are essential for IT and security teams to prioritize and address vulnerabilities.

7. Scanning Frequency: Organizations often perform regular Nessus scans to continuously monitor and improve their security posture. This is crucial for staying ahead of evolving threats and vulnerabilities.

8. Integration: Nessus can be integrated with other security tools and platforms, allowing organizations to automate the scanning process and streamline their vulnerability management.

9. Licensing: Nessus offers both free and commercial versions. The free version is limited in its functionality, while the commercial version provides more features and support.

10. User-Friendly Interface: Nessus offers an easy-to-use web-based interface, making it accessible to a wide range of users, from security experts to less experienced IT professionals.

It's important to note that Nessus is a valuable tool for identifying vulnerabilities, but it doesn't fix these vulnerabilities itself. It's up to the organization's IT and security teams to take the necessary steps to remediate the identified issues to enhance their security.

S. No	Vulnerability Name	Severity	Plugins
1.	Cross-site scripting (stored)	Low	Web application Scanner
2.	Broken Access Control	Medium	SSH weak HMAC Algorithms Supported
3.	SQL Injection	Medium	N/A
4.	Broken Authentication	Low	HSTS Missing from Server
5.	Insecure Direct Object Reference	Info	Service Detection
6.	Security Misconfiguration	Low	SSL cipher Block Chaining cipher suites

7.	Cross-Site Request Forgery	Low	HTTP Form- Based Authentication Detected
8.	Cleartext Transmission of Sensitive Information	Medium	Cleartext Transmission of Sensitive Information over HTTP
9.	ClickJacking	Info	HTML Rendering Layer Vulnerability
10.	webserver allows password auto-completion	Low	SSL/TLS Diffie-Hellman Modulus<=1024 bits(Logjam)

Target Website: testfire.net

Target IP Address: 54.83.41.36

Report:

1. VULNERABILITY NAME: Cross site scripting (stored)

   SEVERITY: Low

   PLUGIN: web application scanner

   PORT: N/A

   DESCRIPTION: It occurs when a malicious script is injected directly into a vulnerable web application. Reflected XSS involves the reflection of a malicious script of a web application, onto a user's browser.

   SOLUTION:
   - When interacting with databases, use parameterized statements or prepared statements to avoid SQL injection, which can be a vector for XSS.
   - Sanitize and validate all user inputs and ensure that any data displayed on the web page properly encoded. Use output encoding libraries or functions to prevent script injection.

   BUSINESS IMPACT: The application stores dangerous data in a database, message forum, visitor log, or other trusted data store. Later, the dangerous data is subsequently read back into the application and included in dynamic content. From an attacker's perspective, the optimal place to inject malicious content is in an area that is displayed to either many users or particularly interesting users. Interesting users typically have elevated privileges in the application or interact with sensitive data that is valuable to the attacker. If one of these users executes malicious content, the attacker may be able to perform privileged operations on behalf of the user or gain access to sensitive data belonging to the user.

2. VULNERABILITY NAME: Broken access control

   SEVERITY: medium

   PLUGIN: SSH weak HMAC Algorithms supported

   PORT: N/A

DESCRIPTION: When access control checks are not applied consistently - or not at all - users are able to access data or perform actions that they should not be allowed to perform. This can lead to a wide range of problems, including information exposures, denial of service, and arbitrary code execution.

SOLUTION:

- Ensure that the application has a robust authentication and authorization mechanism in place. Users should be authenticated before any sensitive actions are performed, and authorization checks should be conducted to verify that users have the necessary permissions for the requested operation.
- Ensure that session management is secure. Use strong session IDs, enforce session timeouts, and regenerate session tokens after login. Implement session fixation protection to prevent session hijacking.

BUSINESS IMPACT: wide-ranging business impact, including financial losses, reputation damage, legal consequences, operational disruption, and loss of customer trust. Addressing this vulnerability is crucial to protect the organization and its stakeholders from these potential negative effects.

3. VULNERABILITY NAME: SQL injection

   SEVERITY: medium

   PLUGIN: N/A (typically would be a security testing tool)

   PORT: port 80/443

   DESCRIPTION: The product constructs all or part of an SQL command using externally influenced input from an upstream component, but it does not neutralize or incorrectly neutralizes special elements that could modify the intended SQL command when it is sent to a downstream component.

   SOLUTION:

   - If parameterized statements are not feasible, use proper input validation and escaping mechanisms to sanitize user inputs before they are used in SQL queries. This helps prevent malicious code injection.
   - Utilize stored procedures for database operations. This can help separate SQL code from application code and reduce the risk of SQL injection.
   - Implement strict input validation for all user inputs, ensuring that data adheres to the expected format and structure.

   BUSINESS IMPACT: In summary, it is crucial to underscore that CWE-89, known as SQL Injection, can exert a profound and diverse business impact. This encompasses critical facets such as data breaches, financial setbacks, harm to reputation, legal ramifications, and operational turmoil. Hence, the imperative of preventing and remedying SQL injection vulnerabilities cannot be overstated, as it is indispensable for fortifying the security and continuity of an organization's applications and data.

4. VULNERABILITY NAME: Broken authentication

   SEVERITY: Low

   PLUGIN: HSTS Missing from HTTPS Server (plugin id:142960)

   PORT: port 443

DESCRIPTION: The system's authorization functionality does not prevent one user from gaining access to another user's data or record by modifying the key value identifying the data.

SOLUTION:

- Implement strong authentication mechanisms, such as multi-factor authentication (MFA) or two-factor authentication (2FA), to ensure that only authorized users can access the system.
- Implement proper authorization controls to ensure that users have the necessary permissions to access specific functions or data. Use role-based access control (RBAC) to manage user privileges effectively.
- Deploy intrusion detection and prevention systems (IDS/IPS) to monitor and block suspicious activities related to authentication and authorization.

BUSINESS IMPACT: To effectively mitigate the business impact of CWE-285, it is imperative that organizations place a strong emphasis on bolstering their authentication and session management practices. This should encompass the adoption of multi-factor authentication, the secure storage of credentials, meticulous session handling, and a commitment to conducting routine security assessments and testing. The rectification of these vulnerabilities stands as a critical imperative, safeguarding sensitive data, user identities, and the organization's overarching security stature and reputation.

5. VULNERABILITY NAME: Insecure Direct object Reference

SEVERITY: Info

PLUGIN: Service Detection (plugin id:22964)

PORT: port 80/443

DESCRIPTION: Insecure Direct Object Reference (IDOR) is a vulnerability that arises when attackers can access or modify objects by manipulating identifiers used in a web application's URLs or parameters. It occurs due to missing access control checks, which fail to verify whether a user should be allowed to access specific data.

SOLUTION:

- Utilize parameterized queries or prepared statements when interacting with databases. This prevents user-controlled input from being executed as SQL code.
- Implement a Content Security Policy to mitigate the risk of cross-site scripting (XSS) attacks, which can be used to execute unauthorized queries.
- Implement account lockout mechanisms that temporarily suspend accounts after a specified number of unauthorized access attempts to prevent brute force attacks.

BUSINESS IMPACT: To effectively mitigate the business impact of CWE-639, organizations must make it a top priority to fortify their access control and authorization mechanisms. This entails the implementation of robust security measures, the regular conduct of comprehensive security assessments, and the deployment of intrusion detection systems to promptly identify and counter unauthorized access attempts. These measures stand as critical in the defence of sensitive data, the overall security of systems, and the preservation of the organization's esteemed reputation.

6. VULNERABILITY NAME: Security Misconfiguration
   SEVERITY: low

   PLUGIN: SSL cipher Block chaining cipher suites

   PORT: port 443

   DESCRIPTION: The product specifies permissions for a security-critical resource in a way that allows that resource to be read or modified by unintended actors.

   SOLUTION:
   - Implement a change management process to document and review all changes to system configurations. This helps prevent unauthorized or unintended alterations.
   - Follow security guidelines provided by software and hardware vendors to ensure secure configurations.
   - Regularly perform compliance checks against industry standards and regulations to ensure that configurations remain in compliance with security requirements.

   BUSINESS IMPACT: To effectively lessen the business consequences stemming from CWE-732, organizations must place a strong emphasis on bolstering their configuration management practices. This involves the establishment of secure default settings, the routine conduct of comprehensive security evaluations, and the deployment of intrusion detection systems to promptly detect and address misconfigurations and vulnerabilities. These actions are of paramount importance in safeguarding sensitive data, fortifying system security, and preserving the organization's esteemed reputation.

7. VULNERABILITY NAME: Cross-Site Request Forgery SEVERITY: Low

   PLUGIN: HTTP Form-Based Authentication Detected

   PORT: port 8080/443

   DESCRIPTION: Cross-Site Request Forgery (CSRF) is an attack that forces an end user to execute unwanted actions on a web application in which they're currently authenticated. With a little help of social engineering (such as sending a link via email or chat), an attacker may trick the users of a web application into executing actions of the attacker's choosing. If the victim is a normal user, a successful CSRF attack can force the user to perform state changing requests like transferring funds, changing their email address, and so forth. If the victim is an administrative account, CSRF can compromise the entire web application.

   SOLUTION:
   - Implement Synchronizer Token Patterns to include unique, randomly generated tokens in each HTTP request and validate these tokens on the server side to verify the authenticity of the request.
   - Utilize the Same Site attribute in cookies to restrict their usage to same-site requests only, preventing them from being sent along with cross-site requests and effectively mitigating CSRF attacks.
   - Utilize framework-specific protections and security features to prevent CSRF attacks. Many modern web frameworks have built-in mechanisms and libraries for handling CSRF vulnerabilities effectively.
   - Implement a Content Security Policy to restrict the sources from which various types of content can be loaded. By specifying the trusted sources of content, you can minimize the risk of malicious code execution and reduce the likelihood of successful CSRF attacks.

BUSINESS IMPACT: To effectively reduce the business repercussions of CWE-352, organizations must prioritize secure coding practices, including the incorporation of anti-CSRF tokens, while also conducting routine security assessments and testing to detect and rectify CSRF vulnerabilities. Furthermore, educating users on secure browsing practices is pivotal in preventing CSRF attacks. These actions are of paramount importance in the protection of data and in upholding the trust of customers and partners.

8. VULNERABILITY NAME: Cleartext Transmission of Sensitive Information

   SEVERITY: Medium

   PLUGIN: Cleartext Transmission of Sensitive Information over HTTP

   PORT: port 443

   DESCRIPTION: The product transmits sensitive or security-critical data in cleartext in a communication channel that can be sniffed by unauthorized actors. Many communication channels can be "sniffed" (monitored) by adversaries during data transmission. For example, in networking, packets can traverse many intermediary nodes from the source to the destination, whether across the internet, an internal network, the cloud, etc. Some actors might have privileged access to a network interface or any link along the channel, such as a router, but they might not be authorized to collect the underlying data. As a result, network traffic could be sniffed by adversaries, spilling security-critical data.

   SOLUTION:
   - Educate users and employees on secure data handling practices, such as recognizing secure websites (look for "https") and avoiding insecure Wi-Fi networks.
   - Employ data masking or redaction techniques to replace sensitive data with placeholders or cryptographic representations during transmission.
   - Implement encryption for data in transit. Use secure encryption protocols such as TLS/SSL to protect sensitive information during transmission.
   - Use secure communication protocols for transmitting data, such as HTTPS for web applications, and ensure that the selected protocols are kept up to date.

   BUSINESS IMPACT: To effectively reduce the business consequences associated with CWE-319, organizations must prioritize the adoption of secure data transmission practices. This includes the utilization of encryption and robust, secure protocols. The routine conduct of security assessments and testing is pivotal in pinpointing and remedying vulnerabilities linked to data transmission. Furthermore, the education of users on secure data handling practices plays a vital role in proactively preventing data exposure incidents. These actions are of paramount importance in the protection of sensitive data and in preserving the trust of both customers and partners.

9. VULNERABILITY NAME: Clickjacking
   SEVERITY: Info

   PLUGIN: HTML Rendering Layer Vulnerability

   PORT: port 80/443

DESCRIPTION: The web application does not restrict or incorrectly restricts frame objects or UI layers that belong to another application or domain, which can lead to user confusion about which interface the user is interacting with.

SOLUTION:

- Set the X-Frame-Options HTTP response header to deny or same-origin to control how your site can be framed. This is supported by most modern browsers.
- Utilize frame-busting JavaScript code in web applications to prevent the embedding of your site within malicious frame's. This code can disrupt clickjacking attempts.
- Implement a Content Security Policy to restrict which domains can embed your site in iframes. This can help prevent unauthorized framing.
- Implement additional security controls to prevent UI redress attacks, such as clickjacking, within your web application.

BUSINESS IMPACT: To effectively reduce the business consequences resulting from CWE-1021, organizations must prioritize the implementation of protective measures, such as frame-busting code. Simultaneously, educating users on safe browsing practices plays a crucial role in preventing clickjacking incidents. Additionally, routine security assessments and testing are pivotal for identifying and mitigating vulnerabilities associated with clickjacking. These actions are of paramount importance in upholding user trust, ensuring data protection, and safeguarding the organization's reputation.

10. VULNERABILITY NAME: web server allows password auto-completion

SEVERITY: Low

PLUGIN: SSL/TLS Diffie-Hellman Modulus <= 1024 Bits (Logjam)

PORT: port 443

DESCRIPTION: Weaknesses in this category are related to the design and implementation of data confidentiality and integrity. Frequently these deal with the use of encoding techniques, encryption libraries, and hashing algorithms. The weaknesses in this category could lead to a degradation of the quality data if they are not addressed.

SOLUTION:

- Implement secure password storage mechanisms, such as using strong and salted cryptographic hashing algorithms like crypt or script.
- Implement robust encryption key management practices, including secure key storage, key rotation, and access controls.
- Utilize well-established and proven cryptographic libraries and algorithms for encryption and decryption.
- Use secure encryption protocols, like TLS/SSL, for data transmission over networks to protect data in transit

BUSINESS IMPACT: To effectively lessen the business consequences associated with CWE-310, organizations must prioritize the adoption of secure cryptographic practices. This entails ensuring proper password storage and robust encryption key management. Simultaneously, conducting routine security assessments and testing is pivotal in detecting and mitigating vulnerabilities related to cryptographic issues. Striving for compliance with pertinent data protection regulations and industry standards is

equally essential. These actions are of paramount importance in the protection of sensitive data, the upholding of user trust, and the preservation of the organization's esteemed reputation.

S. NO	VULNERABILITY NAME	SEVERITY	PLUGINS
1.	Cross-site scripting (DOM based XSS)	Medium	Multiple XSS
2.	jQuery 1.3<3.5.0 Multiple XSS	Medium	CGI Abuse: XSS
3.	Information exposure (web server HTTP header internal IP disclosure)	Low	Webserver HTTP Header Internal IP Disclosure
4.	Information exposure (SSL (Multiple Issues))	Info	SSL Certificate Information
5.	Information exposure (HTTP (Multiple Issues))	Info	HTTP Information
6.	OS identification failed	Info	TCP/IP Remote Probing

TARGET WEBSITE: vit.ac.in

TARGET IP ADDRESS: 122.184.65.22

Report:

1. VULNERABILITY NAME: Cross-site scripting (DOM based XSS)

   SEVERITY: Medium

   PLUGIN: Multiple XSS

   PORT: port 443

   DESCRIPTION: It occurs when a malicious script is injected directly into a vulnerable web application. Reflected XSS involves the reflection of a malicious script of a web application, onto a user's browser. DOM-XSS: DOM stands for Document Object Model and is a programming interface for HTML and XML documents. It represents that programs can change the document structure, style and content. A webpage is a document, and this document can be either displayed in the browser window or as the HTML Source.
   SOLUTION:
   - When interacting with databases, use parameterized statements or prepared statements to avoid SQL injection, which can be a vector for XSS.
   - Sanitize and validate all user inputs and ensure that any data displayed on the web page is properly encoded. Use output encoding libraries or functions to prevent script injection.

- Avoid using DOM manipulation methods that can introduce vulnerabilities. Be cautious with inner HTML, document. Write, and other methods that can execute scripts.
- Consider using well-established JavaScript frameworks and libraries that include built- in security features to prevent DOM XSS.

BUSINESS IMPACT: The application stores dangerous data in a database, message forum, visitor log, or other trusted data store. Later, the dangerous data is subsequently read back into the application and included in dynamic content. From an attacker's perspective, the optimal place to inject malicious content is in an area displayed to many users or particularly interesting users. Interesting users typically have elevated privileges in the application or interact with sensitive data that is valuable to the attacker. If one of these users executes malicious content, the attacker may be able to perform privileged operations on behalf of the user or gain access to sensitive data belonging to the user.

2. VULNERABILITY NAME: jQuery 1.2 < 3.5.0 Multiple XSS

SEVERITY: Medium

PLUGIN: CGI Abuse: XSS

PORT: port 443

DESCRIPTION: It occurs when a malicious script is injected directly into a vulnerable web application. Reflected XSS involves the reflection of a malicious script of a web application, onto a user's browser. Stored XSS (Persistent XSS): In this type of attack, the malicious script is permanently stored on a server or in a database, often in user-generated content like comments or forum posts. When other users view the affected content, the script executes in their browsers, potentially compromising their data or sessions. Reflected XSS: Reflected XSS occurs when the malicious script is embedded in a URL and immediately executed when a victim clicks on a manipulated link. The script is not stored on a server but is reflected off a web application, making it a one-time attack. DOM-based XSS: DOM stands for Document Object Model and is a programming interface for HTML and XML documents. It represents that programs can change the document structure, style and content. A webpage is a document, and this document can be either displayed in the browser window or as the HTML Source.

SOLUTION:

- The most important recommendation is to update to the latest version of jQuery. Newer versions typically include security patches, bug fixes, and performance improvements.
- Stay vigilant about jQuery updates and security announcements. Subscribe to relevant security mailing lists or forums to receive notifications about new releases.
- A Content Security Policy (CSP) can help mitigate various types of client-side vulnerabilities, including Cross-Site Scripting (XSS). Consider implementing a CSP in your web application to restrict the sources from which scripts can be loaded.
- If updating jQuery is not immediately possible, review your code for potential security issues. Ensure that you are escaping or sanitizing user inputs and that you are using safe coding practices to prevent XSS and other security vulnerabilities.

BUSINESS IMPACT: The application stores dangerous data in a database, message forum, visitor log, or other trusted data store. Later, the dangerous data is subsequently read back into the application and included in dynamic content. From an attacker's perspective, the optimal place to inject malicious content is in an area displayed to many users or particularly interesting users. Interesting users typically have elevated privileges in the application or interact with sensitive data

that is valuable to the attacker. If one of these users executes malicious content, the attacker may be able to perform privileged operations on behalf of the user or gain access to sensitive data belonging to the user.

3. VULNERABILITY NAME: Information Exposure (Web Server HTTP Header Internal IP Disclosure)

   SEVERITY: Low

   PLUGIN: Web Server HTTP Header Internal IP Disclosure

   PORT: port 80

   DESCRIPTION: The product exposes sensitive information to an actor that is not explicitly authorized to have access to that information.

   SOLUTION:
   - Carefully review your web server's configuration files (e.g., Apache's httpd.conf, Nginx's nginx.conf) to identify anywhere internal IP addresses are inadvertently included in HTTP response headers.
   - Remove or replace any internal IP addresses in the server configuration files with appropriate placeholders or public IP addresses.
   - If you use proxy servers or load balancers, ensure that they are configured to hide internal IP addresses and only expose public IP addresses instances in HTTP headers.
   - Review and configure HTTP response headers (e.g., Server, X-Powered-By) to limit the exposure of server and technology details, reducing the potential for disclosing internal IP addresses.

   BUSINESS IMPACT: To effectively reduce the business consequences arising from SSL issues, organizations must give utmost priority to establishing a secure SSL/TLS configuration, conducting frequent security assessments and testing. Adhering to data protection regulations and industry standards is imperative. These actions play a pivotal role in safeguarding sensitive data, upholding user trust, and safeguarding the organization's esteemed reputation.

4. VULNERABILITY NAME: Information Exposure (SSL (Multiple Issues))

   SEVERITY: Info

   PLUGIN: SSL certificate Information

   PORT: port 443
   DESCRIPTION: The product exposes sensitive information to an actor that is not explicitly authorized to have access to that information.
   SOLUTION:
   - Configure your web server to use the latest and strongest encryption protocols, such as TLS 1.2 or higher. Disable outdated and insecure protocols like SSLv3.
   - Implement Perfect Forward Secrecy to ensure that even if the private key is compromised, previously recorded encrypted traffic remains secure.
   - Safeguard your private keys by storing them securely and ensuring they are not accessible to unauthorized parties.

BUSINESS IMPACT: To effectively reduce the business consequences arising from SSL issues, organizations must give utmost priority to establishing a secure SSL/TLS configuration, conducting frequent security assessments and testing. Adhering to data protection regulations and industry standards is imperative. These actions play a pivotal role in safeguarding sensitive data, upholding user trust, and safeguarding the organization's esteemed reputation.

5. VULNERABILITY NAME: Information Exposure (HTTP (Multiple Issues))

   SEVERITY: Info

   PLUGIN: HTTP information

   PORT: port 80 /443

   DESCRIPTION: The product exposes sensitive information to an actor that is not explicitly authorized to have access to that information

   SOLUTION:
   - Minimize the use of unnecessary redirects as they can increase page load times. Use 301 (permanent) redirects for consistent content.
   - Regularly check for broken links (404 errors) and fix them to ensure users find the content they are looking for.
   - Properly configure CORS to control which domains are allowed to make requests to your server.
   - Migrate your website from HTTP to HTTPS to ensure secure data transmission. This is essential for user trust and search engine ranking.

   BUSINESS IMPACT: To effectively reduce the business repercussions of HTTP issues, organizations must place a strong emphasis on ongoing monitoring and enhancement of their web services. Consistent performance testing and security assessments, coupled with proactive maintenance, are indispensable to ensure a seamless and secure online presence. Furthermore, offering timely and helpful customer support and transparent communication when HTTP issues arise is instrumental in mitigating adverse effects on user trust and satisfaction.

6. VULNERABILITY NAME: OS Identification Failed

   SEVERITY: Info
   PLUGIN: TCP/IP remote probing

   PORT: N/A

   DESCRIPTION: OS identification failed refers to the inability of a system or software to accurately recognize the operating system running on a device or network. This issue can result from various factors such as misconfigurations, network issues, or compatibility problems. The failure to identify the OS correctly can lead to security vulnerabilities, hindered software updates, and potential operational disruptions. Resolving this issue promptly is crucial to ensure accurate security measures and the smooth functioning of the system or network.

   SOLUTION:

- Minimize the use of unnecessary redirects as they can increase page load times. Use 301 (permanent) redirects for consistent content.
- Regularly check for broken links (404 errors) and fix them to ensure users find the content they are looking for.
- Properly configure CORS to control which domains are allowed to make requests to your server.
- Migrate your website from HTTP to HTTPS to ensure secure data transmission. This is essential for user trust and search engine ranking.

BUSINESS IMPACT: It may lead to operational inefficiencies, potential security vulnerabilities, non-compliance with industry regulations, and a possible loss of customer trust. Such failures could result in downtime, compromised data security, regulatory penalties, and damage to the company's reputation, potentially leading to financial losses and decreased market competitiveness. Resolving this issue promptly is crucial to mitigate these risks and maintain the smooth functioning of the organization.

Stage – 3

Title:

Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

Zero Trust Network Access (ZTNA)

Zero Trust Network Access (ZTNA) is a cybersecurity framework that shifts access control from a network-centric model to an identity-centric model. The foundational principle of ZTNA is "never trust, always verify," meaning that no user or device is trusted by default, regardless of whether they are connecting from inside or outside the corporate network. Unlike traditional remote access solutions that grant users broad access to the entire network, ZTNA creates a secure, application-specific tunnel between the user and the resources they are authorized to access. This ensures that the application remains invisible to unauthorized users and the broader internet, effectively shrinking the attack surface and eliminating the risk of lateral movement across the network.

Zscaler Private Access (ZPA)

Zscaler Private Access (ZPA) is a cloud-delivered ZTNA solution that provides identity- and contextbased access to private applications without granting network-level access. ZPA operates on a "serviceinitiated" model, where users connect to the Zscaler cloud, and the cloud brokers a secure connection to the specific application via ZPA App Connectors deployed in the data center or cloud environment. Because ZPA never places the user on the corporate network, internal IP addresses and network infrastructure are completely hidden. Access policies are dynamically enforced based on user identity, group membership, device posture, and location, ensuring that each user sees only the applications they are explicitly permitted to use. Principle of Least Privilege

The principle of least privilege (PoLP) is a security concept in which a user, device, or process is granted only the minimum levels of access—or permissions—necessary to perform its function. In the context of network security, this means moving away from "fat" network access (where authenticated users can reach large segments of the network) to "thin," application-specific access. By enforcing least privilege through ZPA, organizations ensure that even if a user's credentials are compromised, the attacker cannot pivot to other systems because the network itself is invisible and inaccessible. This principle is foundational to reducing insider threats and limiting blast radius during a security incident.

Identity-Aware Access

Identity-aware access is a security model that bases access decisions on who the user is, rather than where they are connecting from. ZPA integrates directly with existing identity providers (IdPs) such as Microsoft Active Directory, Azure AD, Okta, or Ping Identity. This integration allows security teams to leverage existing user attributes, group memberships, and roles to build granular access policies. For example, a policy can state that only members of the "Finance" group, using a company-managed device, can access the "SAP Finance" application during business hours. This ensures that access is tightly coupled with the user's identity and context, reducing reliance on static IP addresses or network segments.

Application-Specific Access Policies

Application-specific access policies are the core enforcement mechanism within ZPA. Instead of creating rules that allow traffic to entire subnets or IP ranges, administrators define policies that target individual applications or groups of applications. These policies are written in plain language logic and can incorporate multiple conditions, including user identity, device posture, location, and time of day. ZPA allows for the segmentation of internal applications—such as HR systems, internal wikis, development servers, and database interfaces—so that each application has its own unique access policy. This micro-segmentation ensures that a user authenticated for one application has no visibility or connectivity to any other application.

ZPA Architecture and Components

The ZPA architecture is composed of several key components that work together to provide secure, scalable access:

1. Zscaler Cloud (Service Edge): A globally distributed cloud platform that acts as the broker between users and applications. It handles authentication, policy enforcement, and connection brokering without sitting in the data path.
2. App Connectors: Lightweight software VMs or containers deployed within the private data center or cloud environments. These connectors initiate outbound connections to the Zscaler cloud, eliminating the need for inbound firewall rules. They establish secure tunnels to the ZPA edge and forward traffic to the internal applications.
3. ZPA Client Connector: A software client installed on end-user devices (Windows, macOS, Linux, iOS, Android). It authenticates the user, checks device compliance, and establishes a micro-tunnel to the ZPA edge for application access.
4. Application Segments: Logical groupings of applications defined within the ZPA admin console. These segments represent the private resources that users need to access.

Implementation Methodology

The deployment of ZPA follows a structured, phased approach to ensure minimal disruption and maximum security efficacy:

- Phase 1: Discovery and Inventory: Identify all internal applications currently in use. Categorize them by criticality, department, and user base. Tools like network scanners, configuration management databases (CMDB), and interviews with department heads are used to build a comprehensive application inventory.
- Phase 2: Connector Deployment: Deploy ZPA App Connectors in the data center or cloud environments where the applications reside. This involves provisioning virtual machines (using VMware, Hyper-V, or cloud-specific images) and configuring them to connect to the ZPA cloud.
- Phase 3: Application Onboarding: Define application segments within the ZPA console. This includes specifying the IP addresses and ports of the internal applications and associating them with the appropriate App Connector groups.
- Phase 4: Policy Creation: Integrate ZPA with the corporate identity provider (e.g., Azure AD). Create granular access policies based on user groups, device posture, and location. Policies are tested in a simulation mode before enforcement.
- Phase 5: Pilot and Rollout: Select a pilot group of users to test the new access method. Gather feedback, monitor performance, and refine policies. Gradually migrate user populations from legacy VPN access to ZPA.
- Phase 6: VPN Decommissioning: Once all users and applications are migrated, the legacy VPN infrastructure is decommissioned, eliminating a significant attack surface.

Tools and Technologies

- Primary Platform: Zscaler Private Access (ZPA) – The core Zero Trust access broker.
- Identity Provider (IdP): Microsoft Azure Active Directory (AD), Okta, or Ping Identity for user authentication and group synchronization.
- Endpoint Protection Platform (EPP): Integration with endpoint security tools (e.g., CrowdStrike, SentinelOne, Microsoft Defender for Endpoint) to assess device posture and hygiene before granting access.
- Security Information and Event Management (SIEM): Integration with SIEM platforms (e.g., IBM QRadar, Splunk, LogRhythm) for centralized logging and correlation of access events.
- Virtualization Platforms: VMware vSphere, Microsoft Hyper-V, or public cloud environments (AWS, Azure, GCP) for hosting ZPA App Connectors.
- Infrastructure as Code (IaC): Tools like Terraform or Ansible for automated deployment and configuration of App Connectors at scale.
- Digital Experience Monitoring (DEM): Tools to monitor end-user connection quality and application performance.

Integration with Identity Providers

Integrating ZPA with the organization's existing identity provider is a critical step in establishing identity-aware access. This integration is typically done via SAML (Security Assertion Markup Language) or LDAP (Lightweight Directory Access Protocol). Once integrated, ZPA synchronizes user attributes and group memberships, allowing policies to reference these groups directly. For example, when a user authenticates via the ZPA Client Connector, they are redirected to the corporate IdP for credentials and multi-factor authentication (MFA). Upon successful authentication, ZPA receives identity attributes that are then used to evaluate access policies and determine which applications the user is allowed to see and connect to. Device Posture Checking

Beyond user identity, ZPA can enforce policies based on the security posture of the connecting device. Through integration with endpoint detection and response (EDR) and mobile device management (MDM) solutions, ZPA can assess factors such as:

- Antivirus software status and definition updates
- Disk encryption status (e.g., BitLocker, FileVault)
- Operating system patch level
- Presence of jailbreak or root compromise on mobile devices
- Real-time risk score from the EDR tool

If a device fails to meet the defined posture requirements (e.g., antivirus is disabled), ZPA can block access, grant limited access, or trigger a remediation workflow.

Micro-tunneling and Application Visibility

ZPA establishes a secure TLS/DTLS tunnel between the ZPA Client Connector and the ZPA service edge, and another tunnel between the service edge and the App Connector. Crucially, this creates a direct "app-to-user" micro-tunnel. The user's device is never placed on the corporate network; instead, traffic is proxied at Layer 7 (application layer). From the user's perspective, they can only resolve and connect to the specific DNS names or IP addresses of the applications they are authorized to access. All other internal resources remain completely invisible, preventing port scanning, vulnerability reconnaissance, and lateral movement.

Threat Intelligence Integration

To enhance security monitoring, ZPA can be integrated with threat intelligence platforms. Logs generated by ZPA—including connection attempts, policy violations, and user activity—can be forwarded to a SIEM (Security Information and Event Management) system. Within the SIEM, these logs can be correlated with threat intelligence feeds (such as those from MISP) to identify suspicious behavior. For example, if a user's credentials are used to access an application from an IP address associated with known malicious activity, the SIEM can trigger an alert, and integration with ZPA can automatically revoke that user's session. Incident Response in a ZPA Environment

ZPA significantly streamlines the incident response process. When a potential threat is detected, security teams can leverage ZPA's capabilities to:

1. Isolation: Immediately revoke access for a compromised user account or device through the ZPA console, cutting off their connectivity to all internal applications.
2. Forensics: Access detailed logs within ZPA showing exactly which applications were accessed, by whom, from which location, and for how long.
3. Containment: Because users have no network-level access, a compromised endpoint cannot be used to scan or attack other internal systems, containing the blast radius.
4. Granular Blocking: Temporarily block access to specific high-risk applications while allowing access to others, ensuring business continuity during an investigation.

Understanding ZPA: Key Takeaways

Zscaler Private Access represents a fundamental shift in how remote access is architected and secured. Here are the key aspects to understand about the solution:

1. It is cloud-native: ZPA is delivered entirely from the Zscaler cloud, eliminating the need for hardware appliances and complex VPN head-end management.
2. It is identity-centric: Policies are built around users and groups, not IP addresses, making access control more intuitive and aligned with organizational structure.
3. It ensures network invisibility: By never placing users on the network, ZPA renders internal infrastructure invisible to the internet, preventing reconnaissance and DDoS attacks.
4. It is scalable: The cloud architecture allows ZPA to scale instantly to accommodate thousands of concurrent users without performance degradation.
5. It supports hybrid environments: ZPA works seamlessly for applications hosted in on-premises data centers, private clouds, and public cloud environments.

6. It improves user experience: Users get direct, optimized connections to applications without backhauling traffic through a centralized VPN concentrator, reducing latency.

Conclusion:

Stage 1: In conclusion, web application testing is a crucial process that ensures the security, functionality, and reliability of web-based software. It involves a comprehensive evaluation of various aspects, including user interface, functionality, performance, and security to identify and rectify issues before they impact end-users. This process not only enhances the quality of web applications but also bolsters their resilience against potential threats and vulnerabilities. In an age where online services and applications are integral to our daily lives, effective web application testing is essential to deliver a seamless and secure user experience.

Stage 2: The Nessus report for the specific website has highlighted key security insights. It identified vulnerabilities, categorized by severity, with a focus on critical and high-risk issues that need immediate attention. Recommendations for addressing these vulnerabilities include patching, reconfiguration, and other security measures, with a priority on actions that can have the most significant impact. Compliance with industry standards and best practices is considered, ensuring alignment with recognized security benchmarks like OWASP guidelines. The report underscores the need for continuous monitoring, regular updates, and ongoing security efforts to maintain a strong security posture. In summary, the Nessus report offers a clear view of the website's security state, provides guidance on mitigating vulnerabilities, and stresses the importance of sustained vigilance against evolving threats.

Stage 3:

The implementation of Zscaler Private Access marks a pivotal shift from a vulnerable, networkcentric security model to a resilient, identity-centric Zero Trust architecture. By enforcing leastprivilege access and rendering private applications invisible to the internet, this project effectively neutralizes the threats

of lateral movement and network-based attacks. The solution not only fortifies the organization's security posture against modern cyber threats but also delivers a seamless and high-performance experience for a distributed workforce. Furthermore, the decoupling of access from the network simplifies IT operations, reduces dependency on legacy VPN infrastructure, and provides granular visibility into user-application interactions. Ultimately, this initiative establishes a future-ready foundation for secure digital transformation..

Future Scope:

Stage 1: future scope of web application testing

The future scope of web application testing is promising and evolving rapidly, driven by the continuous growth of the internet and web-based applications.

Here are some key aspects of its future scope:

- Increased Demand: As more businesses and services move online, the demand for web applications continues to grow. This, in turn, increases the demand for web application testing to ensure security, reliability, and functionality.
- Advanced Technologies: The complexity of web applications is rising with the integration of advanced technologies like AI, IoT, and blockchain. Testing these cutting-edge applications will require specialized skills and tools.
- Security Emphasis: With the growing number of cyber threats and data breaches, security testing for web applications will become even more critical. Testers will need to focus on vulnerabilities, data protection, and compliance with regulations like GDPR.
- Mobile and Cross-Browser Testing: Mobile usage is on the rise, and web applications must be responsive and functional on various devices and browsers. Testing for cross-compatibility will be essential.
- Performance Testing: As user expectations for speed and performance increase, load testing, stress testing, and other performance testing types will be crucial to ensure web applications can handle traffic spikes.

Stage 2: future scope of the testing process you understood.

The future scope of the web application testing process, particularly in terms of identifying vulnerabilities, is exceedingly promising due to the continuous growth of digital platforms, the escalating importance of cybersecurity, and the ever-evolving threat landscape. Here are some key aspects that highlight the prospective scope of web application testing for vulnerability discovery:

- AI-Driven Testing: The integration of artificial intelligence (AI) and machine learning (ML) into web application testing tools will revolutionize the way vulnerabilities are identified. AI algorithms can swiftly scan vast amounts of code and data to pinpoint vulnerabilities that might elude human testers.
- IoT and Mobile App Testing: With the proliferation of Internet of Things (IoT) devices and the increasing usage of mobile applications, the scope of web application testing will extend to encompass these areas. Ensuring the security of IoT devices and the interfaces they use will become critical.
- Blockchain and Cryptocurrency Security: As blockchain technology and cryptocurrencies continue to gain traction, security testing will be imperative to safeguard transactions and digital assets. Ensuring the robustness of blockchain-based applications will be a specialized niche in web application testing. Serverless Architectures: With the adoption of serverless computing, web application testing will need to adapt to assess the security of serverless functions and APIs. Ensuring that serverless components are free from vulnerabilities will be paramount.

- Container Security: As containerization technologies like Docker and Kubernetes become ubiquitous, web application testing will extend to cover container security. This includes ensuring that container images and orchestration configurations are free from vulnerabilities.

Stage 3: future scope of Zero Trust

The successful deployment of ZPA lays the groundwork for a broader, more integrated security ecosystem. Future enhancements can be explored to further mature the organization's security capabilities:

- Integration with Zscaler Internet Access (ZIA): Converging Private Access (ZPA) with Internet Access (ZIA) to create a unified Secure Access Service Edge (SASE) platform, managing all web and private traffic through a single policy framework.
- Advanced Device Posture Checking: Moving beyond basic compliance to incorporate real-time risk scores from EDR tools, blocking access if a device shows signs of active compromise.
- Privileged Remote Access: Extending ZPA to third-party vendors and contractors with time-bound, audited access to specific critical systems, replacing jump boxes and bastion hosts.
- IoT/OT Segmentation: Applying the same micro-tunneling principles to Internet of Things (IoT) and Operational Technology (OT) devices to ensure they can only communicate with specific management servers.
- AI-Driven Anomaly Detection: Leveraging machine learning on ZPA logs to detect behavioral anomalies, such as a user accessing an application at an unusual hour or from a strange location, triggering automated MFA challenges or access revocation.
- Cloud Workload Protection: Utilizing ZPA to secure connections between containers and microservices in multi-cloud environments (AWS, Azure, GCP), ensuring east-west traffic between workloads is encrypted and authenticated.
- Automated Provisioning/Deprovisioning: Enhancing the identity provider integration to automatically revoke application access the moment an employee is terminated or changes roles, ensuring near real-time access control.
- Digital Experience Monitoring (DEM): Implementing DEM tools to proactively monitor the enduser experience, identifying network latency or application performance issues before they impact productivity.

Topics Explored:

1. Zscaler
2. Zero trust
3. ZPA
4. VPN
5. MISP
6. College Network Information
7. Deployment of SOC in the College
8. Threat Intelligence
9. Incident Response

Tools Explored:

- Nessus Scanner